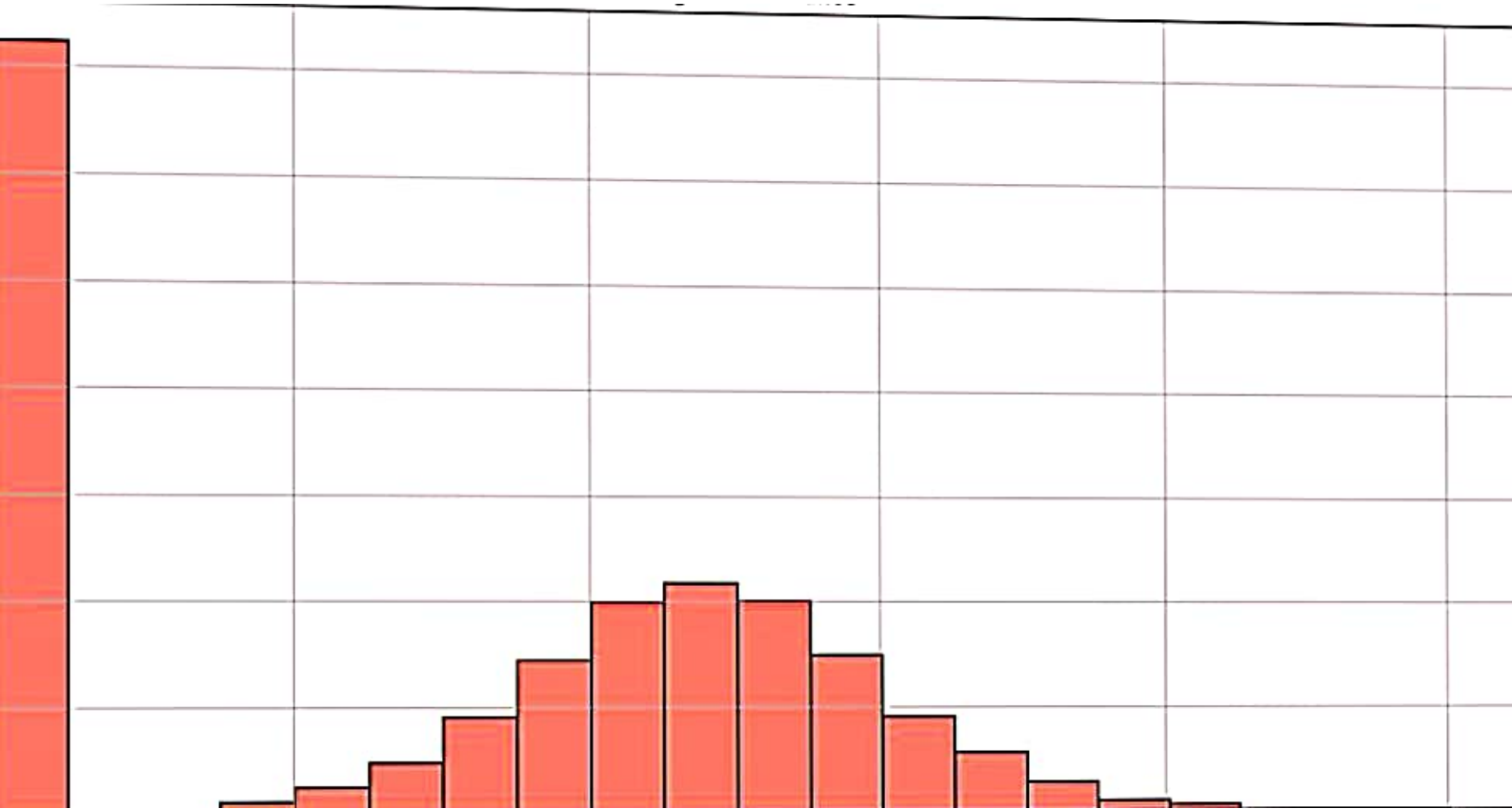
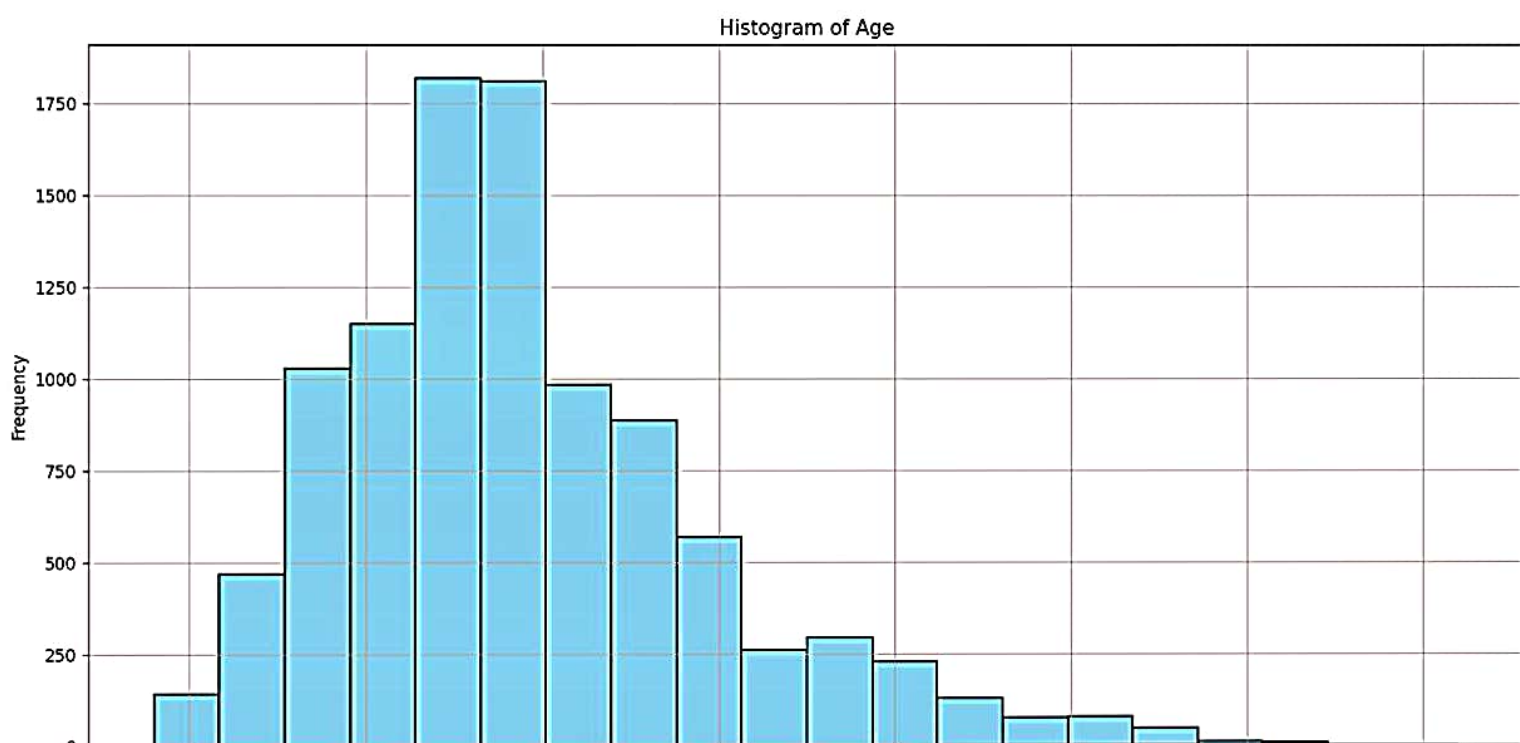


```
# Plot histogram of Age
if 'Age' in df.columns:
    plt.hist(df['Age'].dropna(), bins=20, color='skyblue', edgecolor='black')
    plt.title('Histogram of Age')
    plt.xlabel('Age')
    plt.ylabel('Frequency')
    plt.grid(True)
    plt.show()

# Plot histogram of Balance
if 'Balance' in df.columns:
    plt.hist(df['Balance'].dropna(), bins=20, color='salmon', edgecolor='black')
    plt.title('Histogram of Balance')
    plt.xlabel('Balance')
    plt.ylabel('Frequency')
    plt.grid(True)
    plt.show()
```





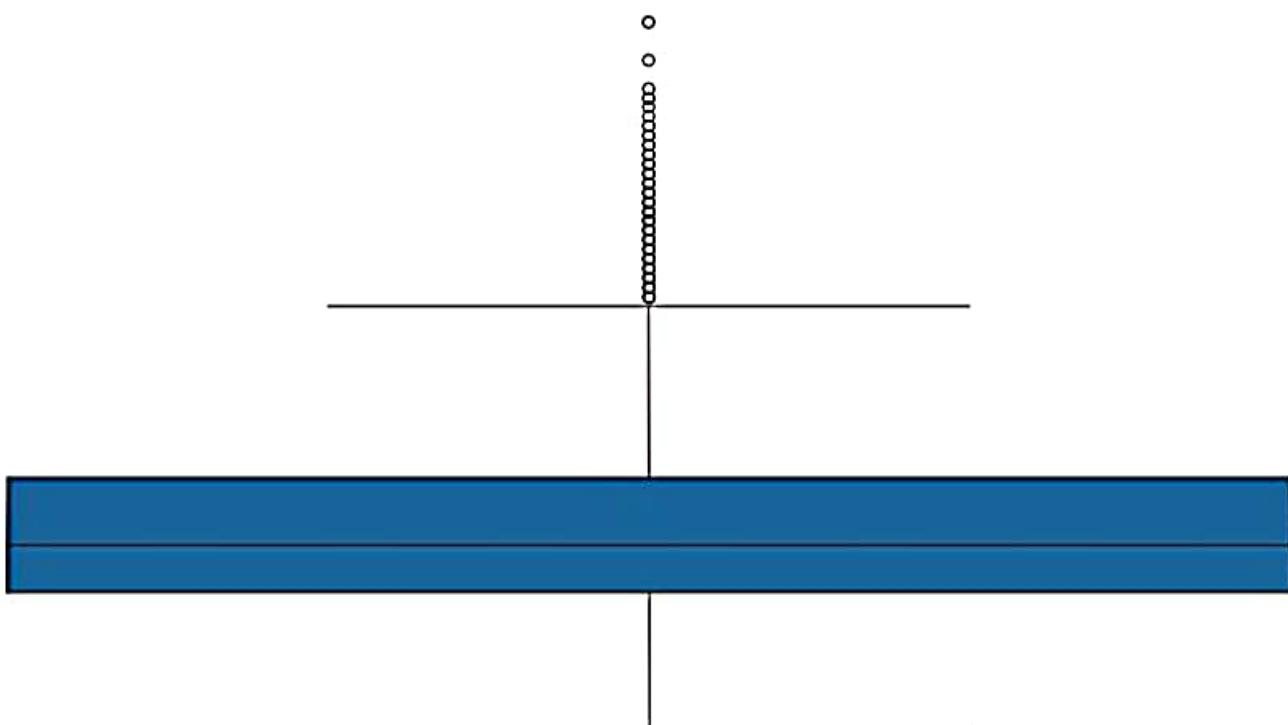
```
# Compute the correlation matrix for numerical columns
correlation_matrix = df.corr(numeric_only=True)

# Plot the correlation heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap="coolwarm", linewidths=0.5)
plt.title("Correlation Heatmap")
plt.show()
```

1.00	0.01	0.01	-0.01	-0.01	0.02	-0.01	0.00	0.02	-0.01
0.01	1.00	-0.00	-0.00	0.01	0.01	-0.00	0.03	-0.00	-0.03
0.01	-0.00	1.00	-0.01	0.03	-0.00	-0.01	0.09	-0.01	0.29
-0.00	-0.00	-0.01	1.00	-0.00	0.01	0.02	-0.00	0.01	-0.01
-0.00	0.01	0.03	-0.01	1.00	-0.30	-0.00	-0.00	0.01	0.12
0.02	0.01	-0.00	0.01	-0.30	1.00	0.00	0.01	0.01	-0.05
-0.00	-0.00	-0.00	0.02	-0.00	0.00	1.00	-0.00	-0.00	-0.01
0.00	0.03	0.09	-0.00	-0.00	0.01	-0.00	1.00	-0.00	-0.16
0.02	-0.00	-0.01	0.01	0.01	0.01	-0.00	-0.00	1.00	0.01
-0.00	-0.00	0.29	-0.01	0.12	-0.00	-0.00	-0.16	0.01	1.00

```
plt.figure(figsize=(6, 6))
sns.boxplot(data=df, y='Age')
plt.title("Ages")
plt.show()
```

```
# Box plot for CustomerId (not recommended for insights)
plt.figure(figsize=(6, 6))
sns.boxplot(data=df, y='CustomerId')
plt.title("CustomerId")
```



```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# Load the dataset
df = pd.read_csv(r"C:\Users\kodad\Downloads\Bank_Churn(in).csv")

# Display column names to understand structure
print("Available columns:", df.columns)

# Calculate Average Balance
if 'Balance' in df.columns:
    avg_balance = df['Balance'].mean()
    print(f"Average Balance: {avg_balance:.2f}")

# Calculate Average Credit Score
if 'CreditScore' in df.columns:
    avg_credit_score = df['CreditScore'].mean()
    print(f"Average Credit Score: {avg_credit_score:.2f}")
```



```
# Check if necessary columns exist
if 'Exited' in df.columns and 'Balance' in df.columns:
    # Group by 'Exited' and calculate mean balance
    avg_balance_comparison = df.groupby('Exited')['Balance'].mean()

    print("Average Balance Comparison:")
    print("Retained Customers (Exited = 0):", avg_balance_comparison.get(0, "N/A"))
    print("Churned Customers (Exited = 1):", avg_balance_comparison.get(1, "N/A"))
else:
    print("Required columns 'Exited' or 'Balance' not found.")
```

```
= RESTART: C:\Users\kodad\AppData\Local\Programs\Python\Python313\Bank_churn.py
Available columns: Index(['CustomerId', 'Surname', 'CreditScore', 'Geography', '
Gender', 'Age',
      'Tenure', 'Balance', 'NumOfProducts', 'HasCrCard', 'IsActiveMember',
      'EstimatedSalary', 'Exited'],
      dtype='object')
Average Balance: 76485.89
Average Credit Score: 650.53
Customer Count by Gender:
Gender
Male      5457
Female    4543
Name: count, dtype: int64

Customer Count by Geography:
Geography
France      5014
Germany     2509
Spain       2477
Name: count, dtype: int64
Average Balance Comparison:
Retained Customers (Exited = 0): 72745.2967788522
Churned Customers (Exited = 1): 91108.53933726068
```

```
# Select numeric columns for the pair plot
numeric_cols = df.select_dtypes(include=['number']).columns.tolist()

# Create the pair plot with 'NumOfProducts' as hue
sns.pairplot(df[numeric_cols], hue='Age', palette='Set2', corner=True)
plt.suptitle("Pair Plot with Age as Hue", y=1.02)
plt.show()
```

